

Task Management API with React Dashboard

Role: Software Developer

Difficulty: junior

Project Type: fullstack

Duration: 14 days

Focus: api_development

Candidate: Rahul Sharma

Project Overview

Build a task management system exposing a REST API and a Next.js dashboard for CRUD operations.

Technologies

- JavaScript
- TypeScript
- React
- Next.js
- Node.js
- Express.js
- Laravel (PHP)
- Tailwind CSS
- MongoDB

Requirements

- Use existing skills; introduce JWT auth and role-based access; deliver measurable API and UI.

Objectives

- Implement secure REST API
- Create responsive dashboard
- Document endpoints
- Deploy locally
- Show code quality
- Demonstrate testing

Functional Requirements

- Create, read, update, delete tasks
- Assign tasks to users
- Filter by status
- JWT login
- Role-based access
- Export tasks to PDF

Non-Functional Requirements

- RESTful design
- JWT auth
- Input validation
- Error handling
- Responsive UI
- Unit tests

Modules

Auth

User registration and login

- JWT issuance
- Password hashing

Tasks

Task CRUD and assignment

- Create task
- Read tasks
- Update task
- Delete task
- Assign user

Dashboard

React UI for task management

- Task list
- Task form
- Filter UI
- Export button

API

Express routes and controllers

- Route definitions
- Middleware
- Error handling

Database

MongoDB schema and models

- User model
- Task model

Testing

Unit and integration tests

- API tests
- Component tests

Implementation Plan

Setup

- Initialize repo
- Configure Next.js
- Set up Express

Auth

- Create user model
- Implement register/login
- Add JWT middleware

Tasks

- Define task schema
- Create CRUD routes
- Add assignment logic

Dashboard

- Build task list page
- Add form modal
- Integrate API

Export

- Implement PDF export
- Add button

Testing & Docs

- Write tests
- Document API

Evaluation Criteria

API Design (40%)

RESTful endpoints, proper status codes

Security (30%)

JWT auth, role checks, input validation

Code Quality (20%)

Clean, modular, comments

Testing (10%)

Coverage, meaningful tests

Deliverables

- GitHub repo
- API documentation
- Live demo URL
- Test report
- Deployment guide
- Screenshots

Suggested Folder Structure

- /frontend
- /backend
- /docs
- /tests
- /scripts
- /deploy

API Endpoints

POST /api/auth/register

Create user

POST /api/auth/login

Login user

GET /api/tasks

List tasks

POST /api/tasks

Create task

PUT /api/tasks/:id

Update task

DELETE /api/tasks/:id

Delete task

POST /api/tasks/:id/assign

Assign user

GET /api/tasks/export

Export PDF

Database Design

users

Store user credentials

- _id
- email
- password
- role

tasks

Store task data

- `_id`
- `title`
- `description`
- `status`
- `assignedTo`
- `createdBy`
- `createdAt`

Testing Plan

- Unit tests for auth
- Unit tests for tasks
- Integration tests for API
- Component tests for dashboard
- End-to-end flow test
- Coverage report

Candidate Fit

Matches Rahul's JavaScript, TypeScript, React, Next.js, Express, JWT, and MongoDB skills while adding role-based auth and PDF export.